

Shell 概述与环境准备

本章学习目标

- 理解什么是 Shell 以及它在操作系统中的角色
- 了解常见的 Shell 种类及区别
- 掌握虚拟机学习环境的配置与验证
- 学会查看和切换系统默认 Shell

1.1 什么是 Shell?

1.1.1 从生活中理解 Shell

想象一下你去餐厅吃饭的场景：

你（顾客）想要吃一份红烧肉，但你不能直接冲进厨房告诉厨师怎么做。你需要通过**服务员**来传达你的需求——服务员听懂你说的话，然后翻译成厨房能理解的指令，最后把做好的菜端给你。

在计算机的世界里，Shell 就扮演着"服务员"的角色：

餐厅场景	计算机场景
你（顾客）	你（用户）
服务员	Shell
厨房	操作系统内核（Kernel）
点菜内容	命令
做好的菜	命令执行结果

1.1.2 技术定义

Shell 是一个命令行解释器，它接收用户输入的命令，解释后传递给操作系统内核执行，并将执行结果返回给用户。

Shell 的英文原意是"壳"，形象地说明了它是包裹在操作系统内核外面的一层"外壳"，用户通过这个外壳与内核进行交互。

1.1.3 Shell 的两层含义

在实际使用中，"Shell"这个词通常有两层含义：

第一层：命令行解释器（交互式 Shell）

当你打开终端，看到闪烁的光标，输入 `ls`、`cd` 这些命令，这就是在使用交互式 Shell。它就像一个实时翻译官，你说一句，它翻译一句给系统执行。

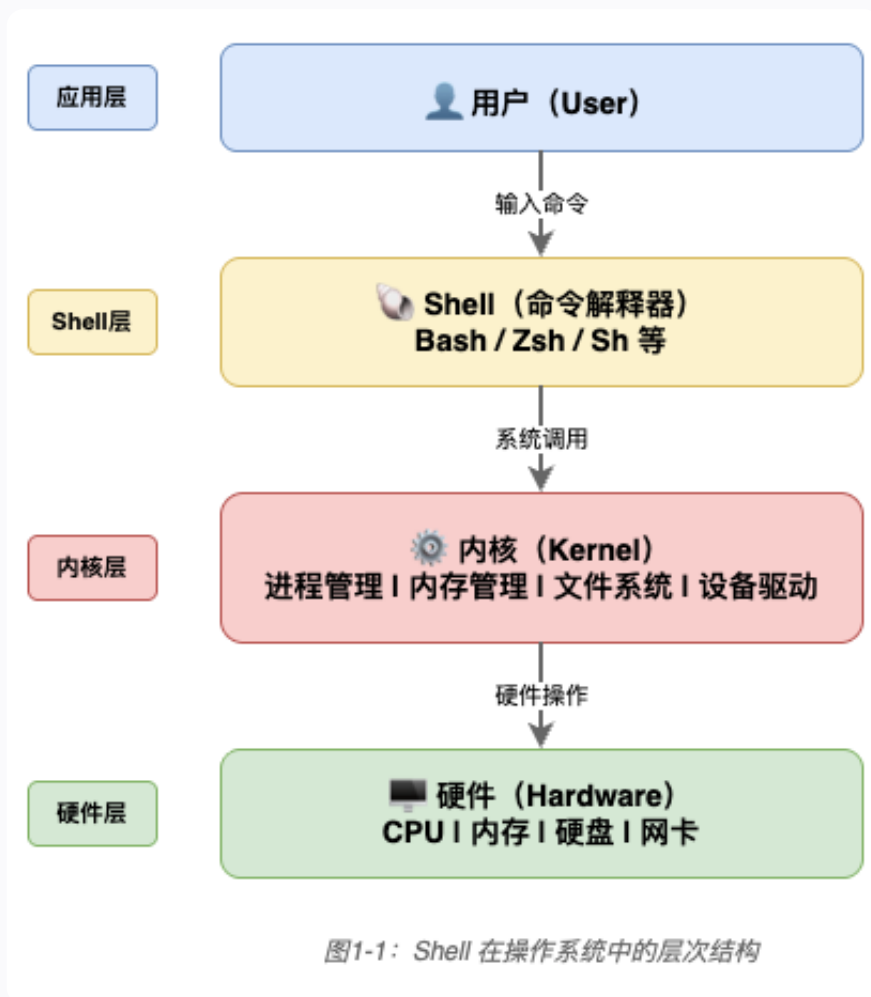
第二层：脚本编程语言（Shell 脚本）

你可以把多条命令写到一个文件里，让 Shell 按顺序自动执行。这个文件就叫做 **Shell 脚本**。这就像把点菜单提前写好，服务员按照单子依次上菜，不用你一道一道地喊。

本课程的重点就是学习如何编写 Shell 脚本，让计算机自动完成重复性的运维工作。

1.2 Shell 在操作系统中的位置

下面这张图展示了用户、Shell、内核和硬件之间的关系：



从图中可以看出，Shell 处于用户和内核之间，起到承上启下的"翻译官"作用。用户不需要了解复杂的内核接口，只需要学会 Shell 命令，就能操控整个计算机系统。

1.3 常见的 Shell 种类

Linux 系统中存在多种 Shell，就像市面上有不同品牌的翻译软件一样，它们功能相似但各有特色。

1.3.1 主流 Shell 对比

Shell 名称	全称	特点	适用场景
sh	Bourne Shell	最古老的 Shell，功能基础	老系统兼容、简单脚本
bash	Bourne Again Shell	sh 的增强版，功能强大，Linux 默认	日常使用、脚本开发（推荐）
zsh	Z Shell	功能最丰富，补全强大	个人终端美化、高级用户
csch	C Shell	语法类似 C 语言	BSD 系统
ksh	Korn Shell	兼容 sh，增加了编程特性	商业 Unix 系统

1.3.2 为什么学习 Bash？

本课程选择 **Bash** 作为教学 Shell，原因如下：

- 第一，普及率最高。** 几乎所有 Linux 发行版（CentOS、Ubuntu、Debian 等）都默认安装 Bash，学会 Bash 就能在绝大多数服务器上工作。
- 第二，功能强大且稳定。** Bash 在兼容传统 sh 语法的基础上，增加了数组、字符串操作、命令补全等实用功能。
- 第三，学习资料丰富。** Bash 拥有最多的教程、文档和社区支持，遇到问题容易找到解决方案。
- 第四，行业标准。** 在企业运维环境中，绝大多数自动化脚本都是用 Bash 编写的。

1.4 查看与切换 Shell

1.4.1 查看系统支持的 Shell

一个 Linux 系统通常会安装多种 Shell，可以通过以下命令查看：

【实操练习 1-2】查看系统支持的 Shell

```
# 查看系统中安装的所有可用 Shell
cat /etc/shells
```

预期输出：

```
/bin/sh
/bin/bash
/usr/bin/sh
/usr/bin/bash
```

这个文件列出了系统中所有可用的 Shell 解释器路径。

1.4.2 查看当前使用的 Shell

有多种方法可以查看当前正在使用的 Shell：

【实操练习 1-3】查看当前 Shell

```
# 方法1：查看 SHELL 环境变量（显示用户默认 Shell）
echo $SHELL

# 方法2：查看当前进程的 Shell（显示实际运行的 Shell）
echo $0

# 方法3：通过进程查看
ps -p $$

# 方法4：查看用户配置文件中的默认 Shell
grep "^$(whoami)" /etc/passwd
```

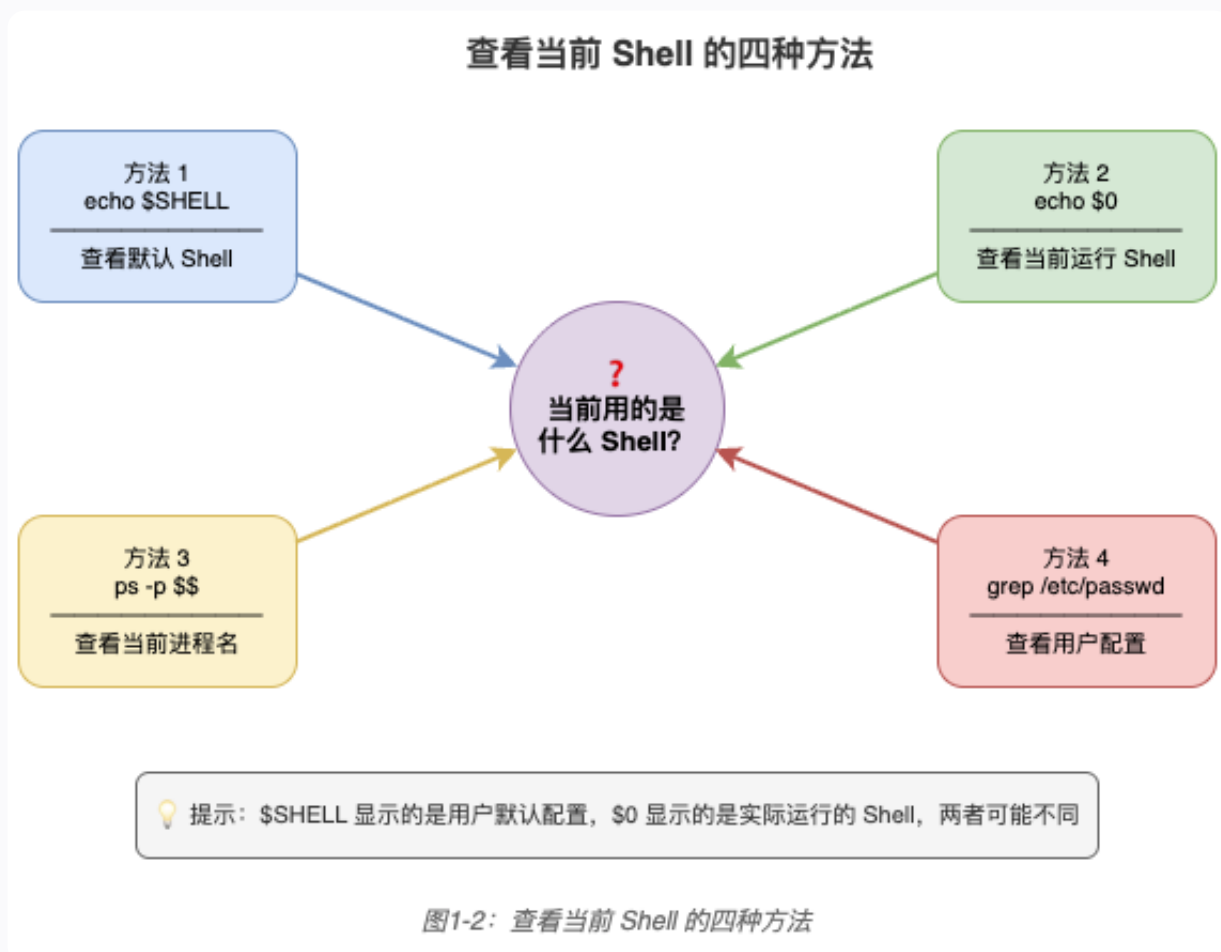
输出解读：

```
# echo $SHELL 输出
/bin/bash          # 说明默认 Shell 是 Bash

# echo $0 输出
-bash              # 当前正在运行的是 Bash (带 - 表示登录 Shell)

# ps -p $$ 输出
  PID TTY          TIME CMD
 1234 pts/0        00:00:00 bash    # 进程名为 bash

# grep 命令输出
root:x:0:0:root:/root:/bin/bash  # 最后一段 /bin/bash 是默认 Shell
```



1.5.3 临时切换 Shell

如果你想临时尝试其他 Shell，可以直接输入 Shell 名称：

【实操练习 1-4】临时切换 Shell

```
# 查看当前 Shell
echo $0

# 切换到 sh
sh

# 确认已切换
echo $0

# 退出 sh, 返回原来的 Shell
exit

# 确认已返回
echo $0
```

执行过程解读：

```
[root@localhost ~]# echo $0
-bash                                # 当前是 bash

[root@localhost ~]# sh
sh-4.2# echo $0
sh                                  # 已切换到 sh

sh-4.2# exit
exit                                # 退出 sh

[root@localhost ~]# echo $0
-bash                                # 返回 bash
```

 **注意：**这种方式只是临时切换，退出后会自动返回原来的 Shell。这就像是你打开了一个新的翻译软件临时用一下，关闭后又回到默认的那个。

1.6 Shell 提示符详解

当你登录 Linux 系统后，会看到一个命令提示符，类似这样：

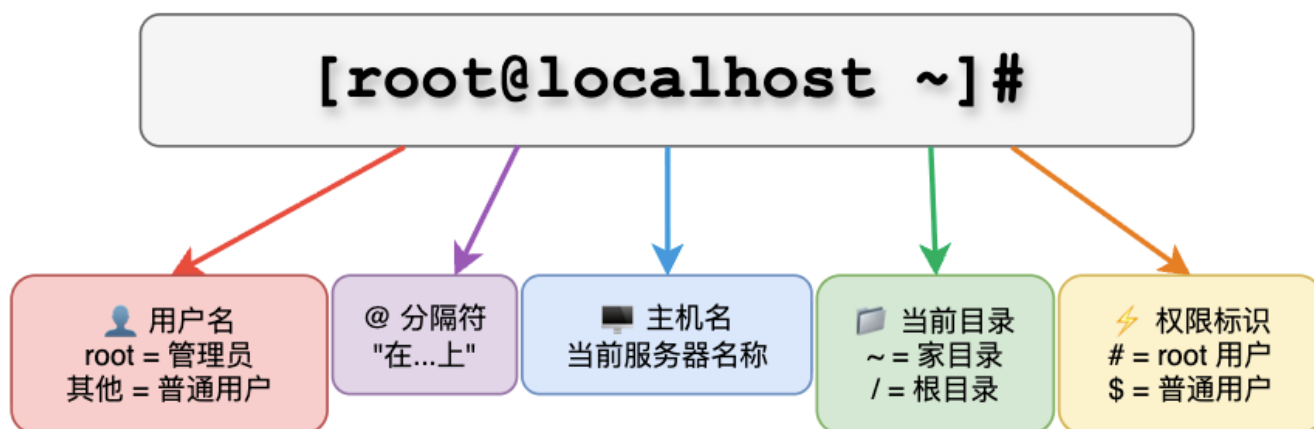
```
[root@localhost ~]#
```

或者

```
[student@centos7 scripts]$
```

这个提示符包含了很多有用的信息：

Shell 提示符结构解析



对比示例

```
[root@localhost ~]#
```

← root 用户，在家目录

```
[student@centos7 scripts]$
```

← 普通用户 student，在 scripts 目录

图1-3: Shell 提示符结构解析

提示符各部分含义：

组成部分	示例	含义
用户名	root / student	当前登录的用户
@	@	分隔符，表示"在...上"
主机名	localhost / centos7	当前服务器的名称
当前目录	~ / scripts	当前所在目录（~代表家目录）
权限标识	# / \$	# 表示 root 用户，\$ 表示普通用户

 **小技巧：**看到 # 就是管理员，看到 \$ 就是普通用户。这个区别很重要，因为很多命令需要管理员权限才能执行。

1.7 本章小结

本章我们学习了以下核心内容：

理论知识：

- 1. Shell 是用户与操作系统内核之间的"翻译官"，负责解释用户命令并传递给内核执行
- 2. Shell 既是命令行解释器，也是一种脚本编程语言
- 3. Linux 系统中有多种 Shell，其中 Bash 是最常用的，也是我们学习的重点
- 4. Shell 提示符包含了用户名、主机名、当前目录和权限标识等重要信息

实践技能：

- 1. 使用 `cat /etc/shells` 查看系统支持的 Shell
- 2. 使用 `echo $SHELL` 和 `echo $0` 查看当前 Shell
- 3. 使用 `bash --version` 查看 Bash 版本
- 4. 临时切换 Shell 和退出的方法